ENTERPRISE SYSTEM AUDIT

Security Audit Report

Authentication, authorization, injection, upload, and secrets-management review — line-by-line verified.

Company Safari Zangu Ventures
Engagement Full-Codebase Enterprise Audit
Date 20 September 2026
Document 4 of 6
Classification Internal — Management & Engineering

Methodology: every finding in this report is drawn directly from the live codebase, database schema, and configuration — file paths and line numbers are cited throughout. No functionality is assumed to exist without direct verification.

1. Security Scorecard

SECURITY SCORE **8.8 / 10**

CSRF, file upload, SQL injection and secrets management all confirmed clean with zero gaps found across an exhaustive check; docked slightly for RBAC/rate-limiting being custom-built rather than framework-hardened, and for no formal encryption-at-rest policy documented

2. Confirmed Strong — Zero Gaps Found

Each of the following was checked exhaustively, not sampled — every form, every POST handler, every user-input-touching query in the codebase was individually traced.

CSRF protection: 0 gaps found across 37 forms and every POST handler GOOD

All 37 <form method="POST"> instances across admin/ and views/ include the CSRF token field. Every admin file that reads $_POST also calls CsrfMiddleware::validateOrDie() a matching number of times — no admin state-changing endpoint found without it. The one endpoint accepting POST without a CSRF check (api/analytics.php, a public page-view beacon) is a deliberate, documented exception protected by rate limiting instead, not a gap.

```
EVIDENCE: 37/37 forms carry CsrfMiddleware::field(); every admin/*.php POST branch calls
validateOrDie(); api/analytics.php:22-26 documents the intentional
rate-limit-instead-of-CSRF design
```

File upload security is solid GOOD

Both upload endpoints (gallery, team photos) require admin auth + CSRF, validate real file content via finfo (not just extension), enforce size caps, regenerate filenames rather than trusting client-supplied names, guard against path traversal with a realpath() containment check, and run every upload through server-side image optimization.

```
EVIDENCE: api/gallery/upload.php:5,17,27-30,44-50,56,65;
api/upload_team_image.php:5,17,30,37-42,52,56-59,65
```

No exploitable SQL injection found GOOD

Every $db->query() call containing a variable was individually traced: filter/sort variables come from hardcoded whitelists (match/ternary against a fixed set of strings), never raw user input; the handful of directly-interpolated integer IDs are explicitly (int)-cast before use. Every WHERE clause built from $_GET/$_POST across the admin list pages uses parameterized placeholders with ->execute($params).

```
EVIDENCE: Full per-file trace across admin/contact_messages.php, customers.php,
Mailer.php, send_single_email.php, team_members.php, and all filterable list pages
```

No hardcoded secrets in committed code GOOD

Every credential constant (DB_PASS, SMTP_PASSWORD, PAYPAL_CLIENT_SECRET, CRON_SECRET) defaults to an empty string via env(), never a literal credential. The disabled PIN-login constant is intentionally randomized with an explicit do-not-replace comment. .env is gitignored and confirmed not tracked in git history.

```
EVIDENCE: config/app.php:84,96,106,133 (empty-string env() defaults); config/app.php:130
(ADMIN_PIN); .gitignore:2; git ls-files check
```

2.1 Additional Controls Already in Place

✓ **CSRF protection** — App\Middleware\CsrfMiddleware applied broadly across admin forms and public forms (contact/book/custom-safari) — token field + validateOrDie() on every POST handler checked this session

✓ **RBAC** — 4-role model (superadmin/admin/staff/agent) with both sidebar-visibility AND data-ownership scoping (agents see only their own leads/quotations/commissions/invoices); staff/admin-only Settings pages have a direct-access guard independent of the nav, not just hidden-but-reachable

✓ **Rate limiting** — Per-IP limiter on contact, book, custom-safari, and track-booking endpoints (3 attempts/15-min lockout on track lookup, shared counter between the JSON lookup and the new PDF-download endpoint)

✓ **Anti-spam** — Honeypot field (off-screen, not display:none, so bots relying on that detection still fall for it) on Contact/Book/Custom-Safari forms, server-side rejection with a fake-success response

✓ **SQL injection** — Consistent use of PDO prepared statements with bound parameters across every user-input-touching query checked this session; no evidence of raw string-concatenated user input into SQL

✓ **Password storage** — bcrypt via PHP's password_hash()/password_verify(), not a weaker/custom hash

✓ **Session cookies** — httponly + samesite=Strict set explicitly (app/bootstrap.php)

✓ **Email authentication (SPF/DKIM/DMARC)** — SPF valid, DKIM confirmed live (hostingermail-a selector, real published key), DMARC upgraded this engagement from p=none to p=quarantine with a reporting address — verified via live DNS queries against multiple public resolvers

✓ **Secrets management** — DB/SMTP credentials read from .env via env() helper, not hardcoded in committed PHP — config/app.php only carries fallback defaults for non-secret business constants

3. Risk Matrix

Risk	Severity	Likelihood	Impact
Stale PARK_FEES config fallback undercharges if DB fails	MEDIUM	Low (requires a DB outage)	Financial — undercharging guests by $10-12/park entry
Broken admin help guide (missing file)	LOW	Certain (always triggers)	Cosmetic/UX only — no data or security impact
Unauthenticated client-side call to Anthropic API in poster maker	LOW	Certain (button always fails)	Feature non-functional; no credential is exposed since none exists client-side
13 admin lists with no pagination limit	LOW	Grows with data volume	Performance degradation only, not a security issue, at current row counts
Cross-table collation mismatch (bookings vs commissions)	LOW	Already mitigated with a workaround	Query correctness risk if the workaround is ever removed without realizing why it's there

4. Security Controls Inventory

Control	Detail	Status
Authentication	Session-based, httponly+samesite=Strict cookies	✓ CONFIRMED
Authorization (RBAC)	4-role model, both nav-visibility AND data-ownership scoping	✓ CONFIRMED
CSRF protection	37/37 forms, all POST handlers	✓ CONFIRMED
Rate limiting	Contact, booking, custom-safari, track-lookup, track-PDF (shared limiter)	✓ CONFIRMED
Anti-spam (honeypot)	3 public forms, off-screen field, fake-success response	✓ CONFIRMED
SQL injection defense	Prepared statements throughout; every variable query manually traced	✓ CONFIRMED
File upload validation	Real MIME sniffing (finfo), size caps, path-traversal guards, regenerated filenames	✓ CONFIRMED
Password storage	bcrypt via password_hash()/password_verify()	✓ CONFIRMED
Secrets management	.env-sourced, empty-string defaults, .env gitignored and not tracked	✓ CONFIRMED
Email authentication (SPF/DKIM/DMARC)	All three verified live via DNS this engagement; DMARC upgraded to enforcing	✓ CONFIRMED
Encryption at rest	No explicit database-level encryption-at-rest policy found or documented	✗ MISSING
API rate limiting (general)	Only specific endpoints (contact/book/custom-safari/track) are limited — no blanket API gateway throttling	■ PARTIAL

ENTERPRISE SYSTEM AUDIT

Security Audit Report

Authentication, authorization, injection, upload, and secrets-management review — line-by-line verified.

Company Safari Zangu Ventures
Engagement Full-Codebase Enterprise Audit
Date 20 September 2026
Document 4 of 6
Classification Internal — Management & Engineering

Methodology: every finding in this report is drawn directly from the live codebase, database schema, and configuration — file paths and line numbers are cited throughout. No functionality is assumed to exist without direct verification.

1. Security Scorecard

SECURITY SCORE **8.8 / 10**

CSRF, file upload, SQL injection and secrets management all confirmed clean with zero gaps found across an exhaustive check; docked slightly for RBAC/rate-limiting being custom-built rather than framework-hardened, and for no formal encryption-at-rest policy documented

2. Confirmed Strong — Zero Gaps Found

Each of the following was checked exhaustively, not sampled — every form, every POST handler, every user-input-touching query in the codebase was individually traced.

CSRF protection: 0 gaps found across 37 forms and every POST handler GOOD

All 37 <form method="POST"> instances across admin/ and views/ include the CSRF token field. Every admin file that reads $_POST also calls CsrfMiddleware::validateOrDie() a matching number of times — no admin state-changing endpoint found without it. The one endpoint accepting POST without a CSRF check (api/analytics.php, a public page-view beacon) is a deliberate, documented exception protected by rate limiting instead, not a gap.

```
EVIDENCE: 37/37 forms carry CsrfMiddleware::field(); every admin/*.php POST branch calls
validateOrDie(); api/analytics.php:22-26 documents the intentional
rate-limit-instead-of-CSRF design
```

File upload security is solid GOOD

Both upload endpoints (gallery, team photos) require admin auth + CSRF, validate real file content via finfo (not just extension), enforce size caps, regenerate filenames rather than trusting client-supplied names, guard against path traversal with a realpath() containment check, and run every upload through server-side image optimization.

```
EVIDENCE: api/gallery/upload.php:5,17,27-30,44-50,56,65;
api/upload_team_image.php:5,17,30,37-42,52,56-59,65
```

No exploitable SQL injection found GOOD

Every $db->query() call containing a variable was individually traced: filter/sort variables come from hardcoded whitelists (match/ternary against a fixed set of strings), never raw user input; the handful of directly-interpolated integer IDs are explicitly (int)-cast before use. Every WHERE clause built from $_GET/$_POST across the admin list pages uses parameterized placeholders with ->execute($params).

```
EVIDENCE: Full per-file trace across admin/contact_messages.php, customers.php,
Mailer.php, send_single_email.php, team_members.php, and all filterable list pages
```

No hardcoded secrets in committed code GOOD

Every credential constant (DB_PASS, SMTP_PASSWORD, PAYPAL_CLIENT_SECRET, CRON_SECRET) defaults to an empty string via env(), never a literal credential. The disabled PIN-login constant is intentionally randomized with an explicit do-not-replace comment. .env is gitignored and confirmed not tracked in git history.

```
EVIDENCE: config/app.php:84,96,106,133 (empty-string env() defaults); config/app.php:130
(ADMIN_PIN); .gitignore:2; git ls-files check
```

2.1 Additional Controls Already in Place

✓ **CSRF protection** — App\Middleware\CsrfMiddleware applied broadly across admin forms and public forms (contact/book/custom-safari) — token field + validateOrDie() on every POST handler checked this session

✓ **RBAC** — 4-role model (superadmin/admin/staff/agent) with both sidebar-visibility AND data-ownership scoping (agents see only their own leads/quotations/commissions/invoices); staff/admin-only Settings pages have a direct-access guard independent of the nav, not just hidden-but-reachable

✓ **Rate limiting** — Per-IP limiter on contact, book, custom-safari, and track-booking endpoints (3 attempts/15-min lockout on track lookup, shared counter between the JSON lookup and the new PDF-download endpoint)

✓ **Anti-spam** — Honeypot field (off-screen, not display:none, so bots relying on that detection still fall for it) on Contact/Book/Custom-Safari forms, server-side rejection with a fake-success response

✓ **SQL injection** — Consistent use of PDO prepared statements with bound parameters across every user-input-touching query checked this session; no evidence of raw string-concatenated user input into SQL

✓ **Password storage** — bcrypt via PHP's password_hash()/password_verify(), not a weaker/custom hash

✓ **Session cookies** — httponly + samesite=Strict set explicitly (app/bootstrap.php)

✓ **Email authentication (SPF/DKIM/DMARC)** — SPF valid, DKIM confirmed live (hostingermail-a selector, real published key), DMARC upgraded this engagement from p=none to p=quarantine with a reporting address — verified via live DNS queries against multiple public resolvers

✓ **Secrets management** — DB/SMTP credentials read from .env via env() helper, not hardcoded in committed PHP — config/app.php only carries fallback defaults for non-secret business constants

3. Risk Matrix

Risk	Severity	Likelihood	Impact
Stale PARK_FEES config fallback undercharges if DB fails	MEDIUM	Low (requires a DB outage)	Financial — undercharging guests by $10-12/park entry
Broken admin help guide (missing file)	LOW	Certain (always triggers)	Cosmetic/UX only — no data or security impact
Unauthenticated client-side call to Anthropic API in poster maker	LOW	Certain (button always fails)	Feature non-functional; no credential is exposed since none exists client-side
13 admin lists with no pagination limit	LOW	Grows with data volume	Performance degradation only, not a security issue, at current row counts
Cross-table collation mismatch (bookings vs commissions)	LOW	Already mitigated with a workaround	Query correctness risk if the workaround is ever removed without realizing why it's there

4. Security Controls Inventory

Control	Detail	Status
Authentication	Session-based, httponly+samesite=Strict cookies	✓ CONFIRMED
Authorization (RBAC)	4-role model, both nav-visibility AND data-ownership scoping	✓ CONFIRMED
CSRF protection	37/37 forms, all POST handlers	✓ CONFIRMED
Rate limiting	Contact, booking, custom-safari, track-lookup, track-PDF (shared limiter)	✓ CONFIRMED
Anti-spam (honeypot)	3 public forms, off-screen field, fake-success response	✓ CONFIRMED
SQL injection defense	Prepared statements throughout; every variable query manually traced	✓ CONFIRMED
File upload validation	Real MIME sniffing (finfo), size caps, path-traversal guards, regenerated filenames	✓ CONFIRMED
Password storage	bcrypt via password_hash()/password_verify()	✓ CONFIRMED
Secrets management	.env-sourced, empty-string defaults, .env gitignored and not tracked	✓ CONFIRMED
Email authentication (SPF/DKIM/DMARC)	All three verified live via DNS this engagement; DMARC upgraded to enforcing	✓ CONFIRMED
Encryption at rest	No explicit database-level encryption-at-rest policy found or documented	✗ MISSING
API rate limiting (general)	Only specific endpoints (contact/book/custom-safari/track) are limited — no blanket API gateway throttling	■ PARTIAL